Guía de ejercicios: principios SOLID (L, I y D)

Esta guía arranca por tres principios: sustitución de Liskov (L), segregación de interfaces (I) e inversión de dependencias (D). El ejercicio integrador del final suma los otros dos (S y O).

Cada ejercicio trae un diseño con problemas, en diagrama de clases y en código C#. Para cada uno tenés que entregar:

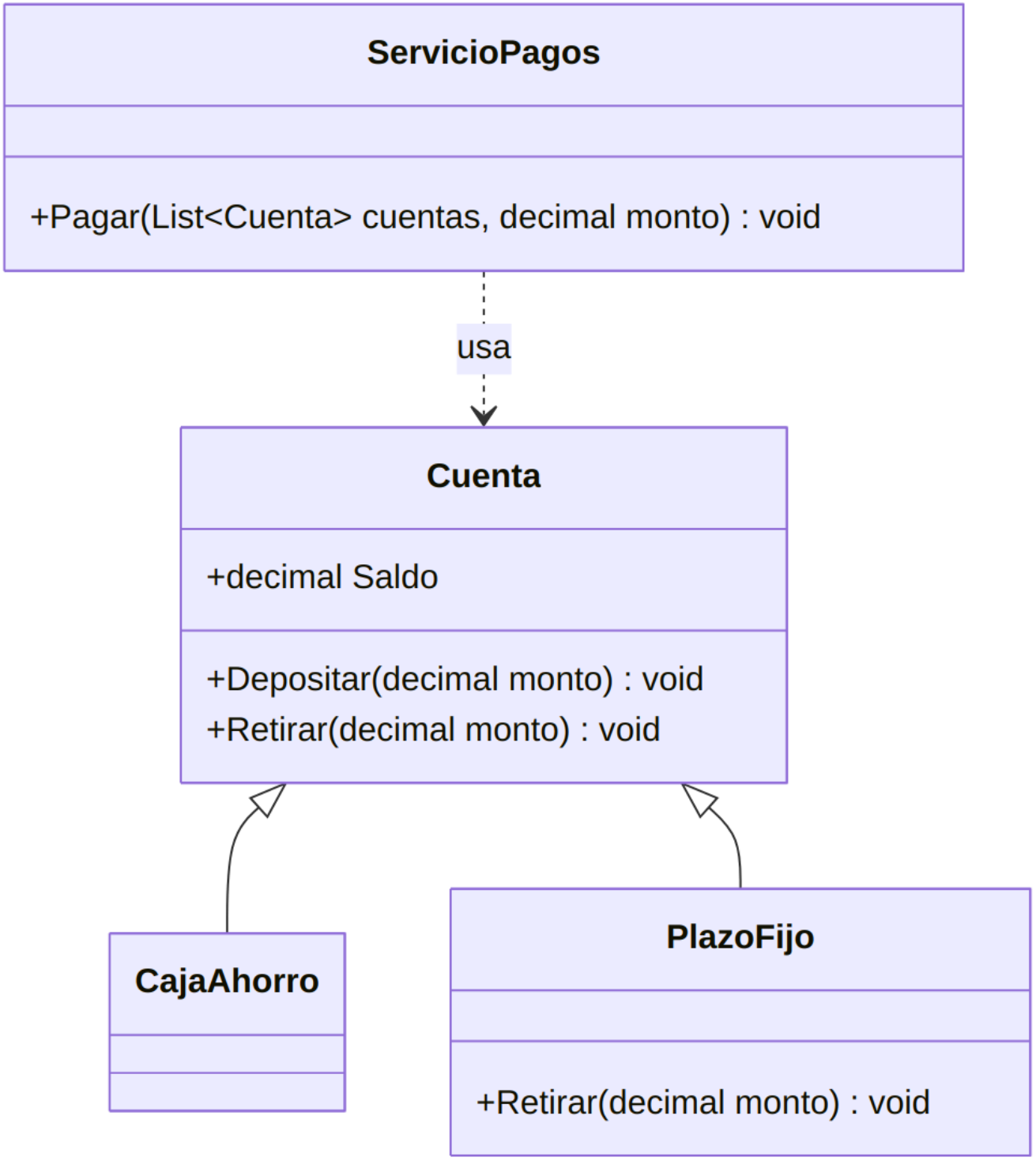
1. Las respuestas a las preguntas de la consigna.
2. El diagrama de clases de tu diseño propuesto.
3. El código C# de tu solución.

No hace falta que el código compile como proyecto completo. Las clases que no se muestran (`Entrega`, `Reporte`, `Nota`, etc.) asumilas como clases simples con las propiedades que necesites.

Parte 1. Principio de sustitución de Liskov

Ejercicio L1. Cuentas bancarias

Un banco modela sus cuentas con una jerarquía. El servicio de pagos recorre las cuentas del cliente y debita el monto de la primera que tenga saldo suficiente.




```
public class Cuenta
{
    public decimal Saldo { get; protected set; }

    public virtual void Depositar(decimal monto)
    {
        Saldo += monto;
    }

    public virtual void Retirar(decimal monto)
    {
```

```
        if (monto > Saldo)
        {
            throw new InvalidOperationException("Saldo insuficiente");
        }
        Saldo -= monto;
    }
}

public class CajaAhorro : Cuenta
{
}

public class PlazoFijo : Cuenta
{
    public override void Retirar(decimal monto)
    {
        throw new NotSupportedException("No se puede retirar antes del
vencimiento");
    }
}

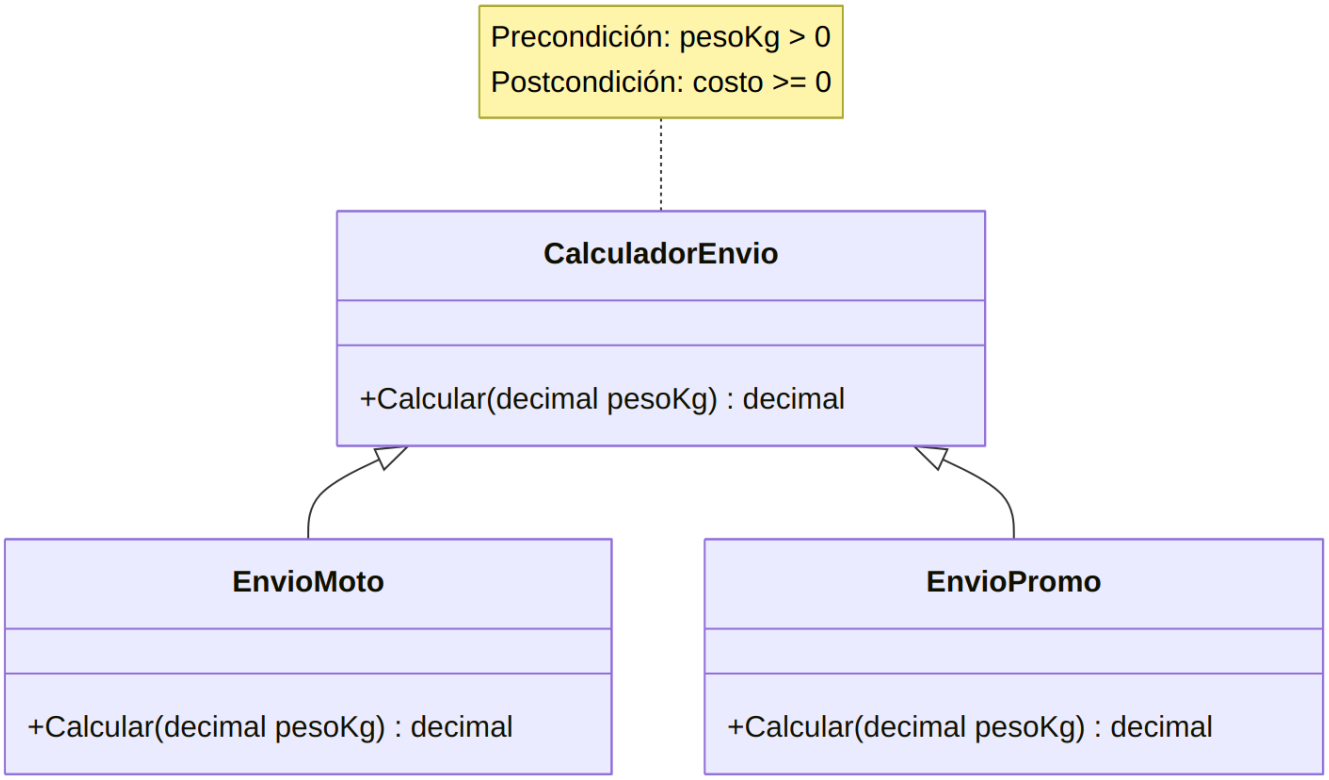
public class ServicioPagos
{
    public void Pagar(List<Cuenta> cuentas, decimal monto)
    {
        foreach (Cuenta cuenta in cuentas)
        {
            if (cuenta.Saldo >= monto)
            {
                cuenta.Retirar(monto);
                return;
            }
        }
    }
}
```

Consigna

1. ¿Qué pasa si la lista incluye un `PlazoFijo` con saldo suficiente?
2. Un compañero propone agregar `if (cuenta is PlazoFijo) continue;` dentro del `foreach` de `ServicioPagos`. ¿Qué principio rompe esa solución además de LSP? Justificá.
3. Rediseñá la jerarquía para que `ServicioPagos` no necesite saber qué tipo de cuenta recibe.
4. En tu diseño, ¿el error de pasarle un plazo fijo a `ServicioPagos` aparece en tiempo de ejecución o en tiempo de compilación?

Ejercicio L2. Cálculo de costo de envío

Una tienda calcula el costo de envío según el peso del paquete. La clase base documenta su contrato en la nota del diagrama.




```
// Contrato de la clase base:
// - Precondición: pesoKg > 0
// - Postcondición: devuelve un costo >= 0
public class CalculadorEnvio
{
    public virtual decimal Calcular(decimal pesoKg)
    {
        if (pesoKg <= 0)
        {
            throw new ArgumentException("Peso inválido");
        }
        return 500 + pesoKg * 100;
    }
}

public class EnvioMoto : CalculadorEnvio
{
    public override decimal Calcular(decimal pesoKg)
    {
        if (pesoKg > 10)
        {
            throw new ArgumentException("La moto no lleva más de 10 kg");
        }
        return 800;
    }
}

public class EnvioPromo : CalculadorEnvio
{
    public override decimal Calcular(decimal pesoKg)
```

```
    {
        return base.Calcular(pesoKg) - 1500;
    }
}
```

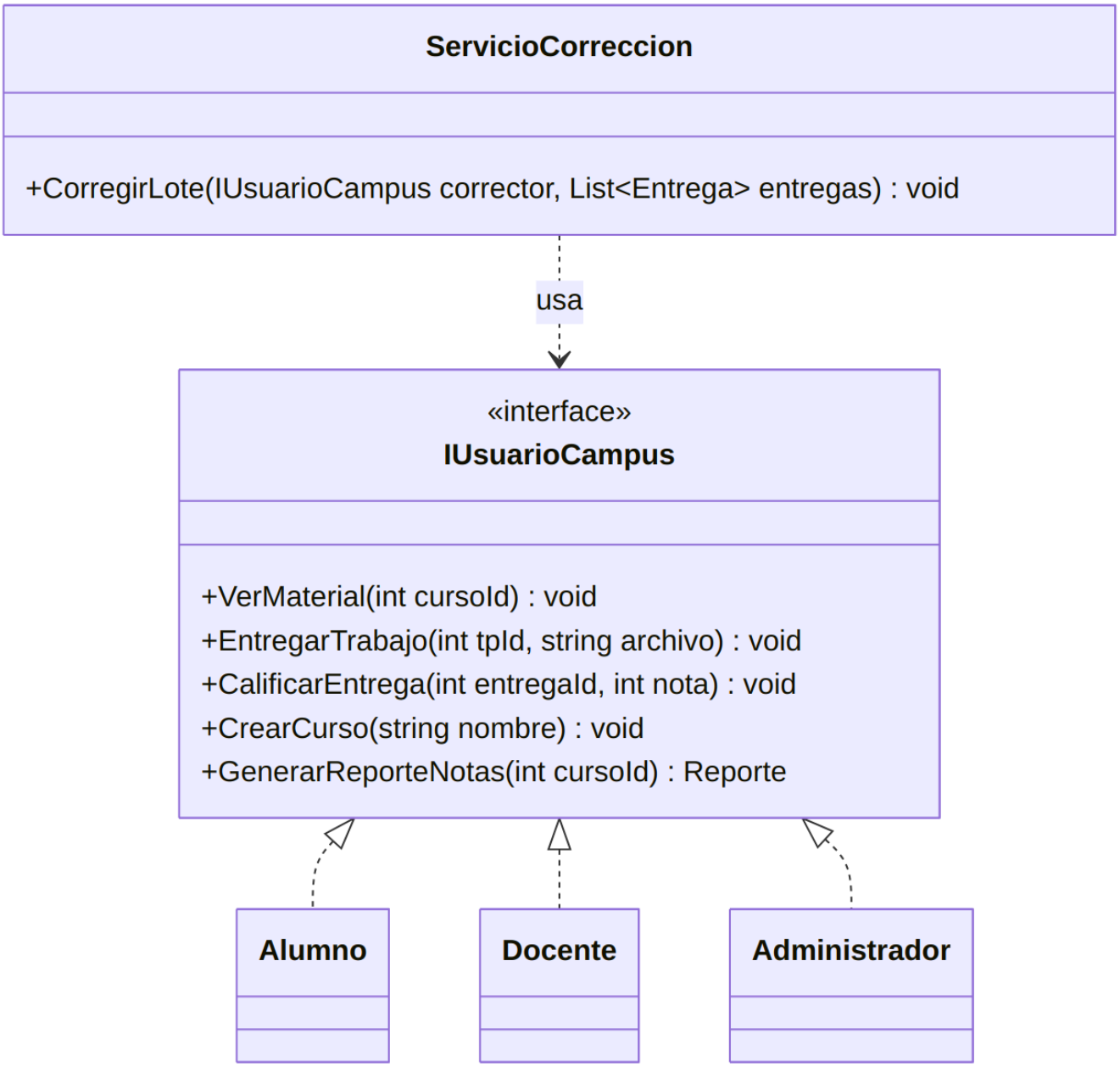
Consigna

1. Para cada subclase, indicá qué parte del contrato de la base rompe: la precondición o la postcondición. Explicá si la fortalece o la debilita.
2. Escribí un método `VerificarContrato(CalculadorEnvio calculador)` que pruebe el contrato de la clase base con varios pesos (por ejemplo 0.5, 5, 15 y 50 kg) y lance una excepción si algo no se cumple. Ejecutalo con las tres clases y anotá qué pasa en cada caso.
3. Proponé un diseño donde las restricciones de cada método de envío formen parte explícita de la abstracción, de modo que ningún cliente se lleve una sorpresa.
4. Pensá si la promoción tiene que ser una subclase o si se puede modelar de otra forma. Justificá tu elección.

Parte 2. Principio de segregación de interfaces

Ejercicio I1. Usuarios del campus virtual

El campus de la facultad tiene una única interfaz para todos los tipos de usuario.




```
public interface IUsuarioCampus
{
    void VerMaterial(int cursoId);
    void EntregarTrabajo(int tpId, string archivo);
    void CalificarEntrega(int entregaId, int nota);
    void CrearCurso(string nombre);
    Reporte GenerarReporteNotas(int cursoId);
}

public class Alumno : IUsuarioCampus
{
    public void VerMaterial(int cursoId)
    {
        // ...
    }

    public void EntregarTrabajo(int tpId, string archivo)
```

```
    {
        // ...
    }

    public void CalificarEntrega(int entregaId, int nota)
    {
        throw new NotImplementedException();
    }

    public void CrearCurso(string nombre)
    {
        throw new NotImplementedException();
    }

    public Reporte GenerarReporteNotas(int cursoId)
    {
        throw new NotImplementedException();
    }
}

public class ServicioCorreccion
{
    public void CorregirLote(IUsuarioCampus corrector, List<Entrega> entregas)
    {
        foreach (Entrega entrega in entregas)
        {
            corrector.CalificarEntrega(entrega.Id, entrega.NotaSugerida);
        }
    }
}
```

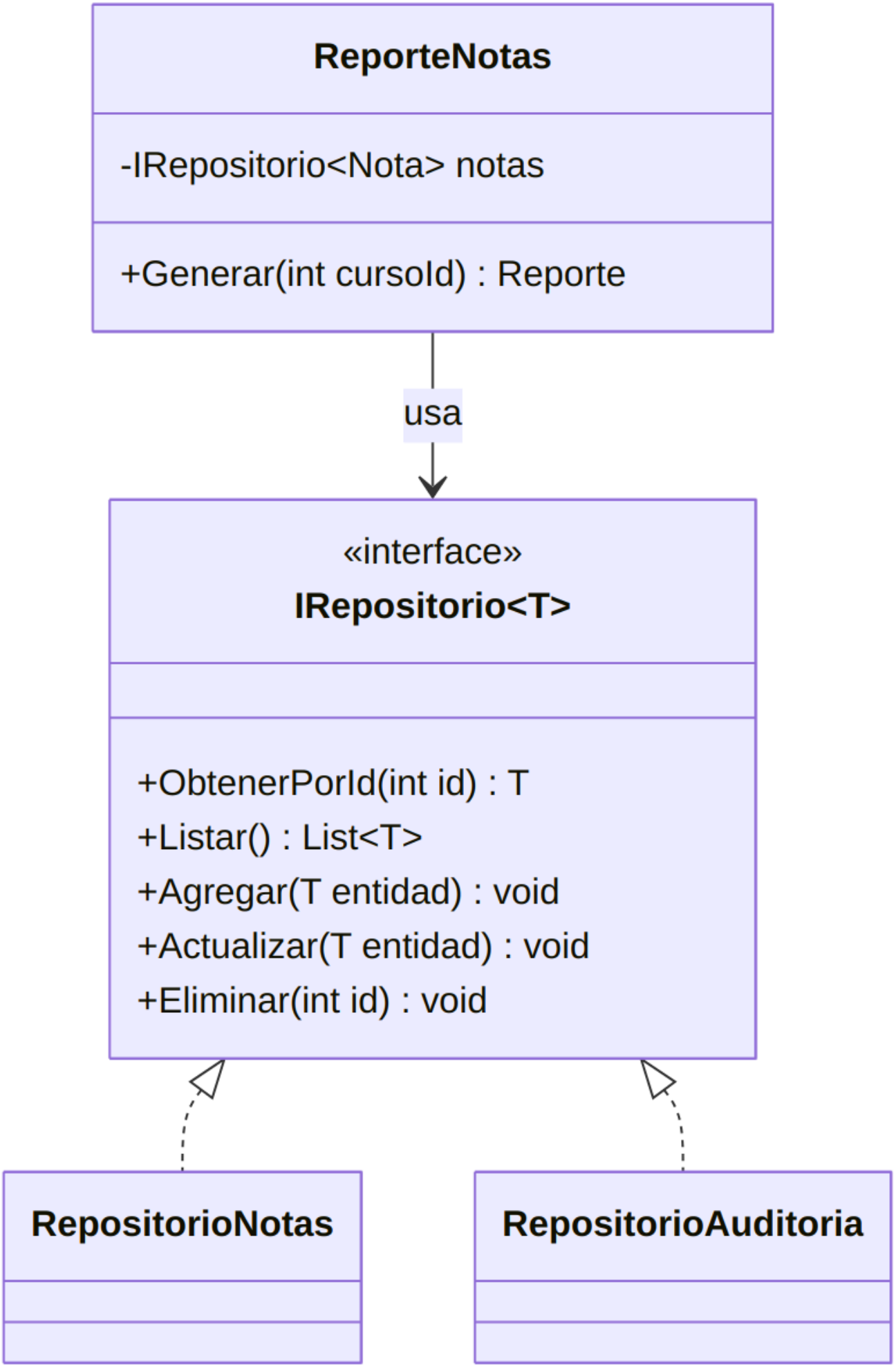
`Docente` y `Administrador` siguen el mismo esquema que `Alumno`: implementan lo que les corresponde y lanzan `NotImplementedException` en el resto.

Consigna

1. Suponiendo que un docente puede ver material y calificar, y un administrador puede crear cursos y generar reportes, ¿cuántos `NotImplementedException` hay entre las tres clases?
2. Cambia la firma de `CalificarEntrega` para que además reciba una devolución escrita. ¿Qué clases hay que modificar? ¿Cuáles de ellas usan ese método de verdad?
3. ¿Qué métodos de `IUsuarioCampus` usa realmente `ServicioCorreccion`? ¿Qué podría hacer con el parámetro `corrector` que no debería poder hacer?
4. Rediseñá las interfaces pensando en quién las consume y no en quién las implementa.
5. Relacioná este ejercicio con el L1. ¿Qué tienen en común un `NotImplementedException` y el `NotSupportedException` de `PlazoFijo`?

Ejercicio I2. Repositorio genérico y auditoría

El sistema usa un repositorio genérico para todas las entidades. Hace poco se agregó un registro de auditoría, que por definición nunca se modifica ni se borra.


ReporteNotas
-IRepositorio<Nota> notas
+Generar(int cursoId) : Reporte
usa
«interface»
IRepositorio<T>
+ObtenerPorId(int id) : T
+Listar() : List<T>
+Agregar(T entidad) : void
+Actualizar(T entidad) : void
+Eliminar(int id) : void
RepositorioNotas
RepositorioAuditoria

```
public interface IRepositorio<T>
{
    T ObtenerPorId(int id);
    List<T> Listar();
    void Agregar(T entidad);
    void Actualizar(T entidad);
    void Eliminar(int id);
}

public class RepositorioAuditoria : IRepositorio<RegistroAuditoria>
{
    public RegistroAuditoria ObtenerPorId(int id)
    {
        // ...
        return null;
    }

    public List<RegistroAuditoria> Listar()
    {
        // ...
        return new List<RegistroAuditoria>();
    }

    public void Agregar(RegistroAuditoria registro)
    {
        // ...
    }

    public void Actualizar(RegistroAuditoria registro)
    {
        throw new InvalidOperationException("La auditoría no se modifica");
    }

    public void Eliminar(int id)
    {
        throw new InvalidOperationException("La auditoría no se borra");
    }
}

public class ReporteNotas
{
    private readonly IRepositorio<Nota> notas;

    public ReporteNotas(IRepositorio<Nota> notas)
    {
        this.notas = notas;
    }

    public Reporte Generar(int cursoId)
    {
        List<Nota> todas = notas.Listar();
        // arma el reporte a partir de la lista
```

```
            return new Reporte();
        }
    }
```

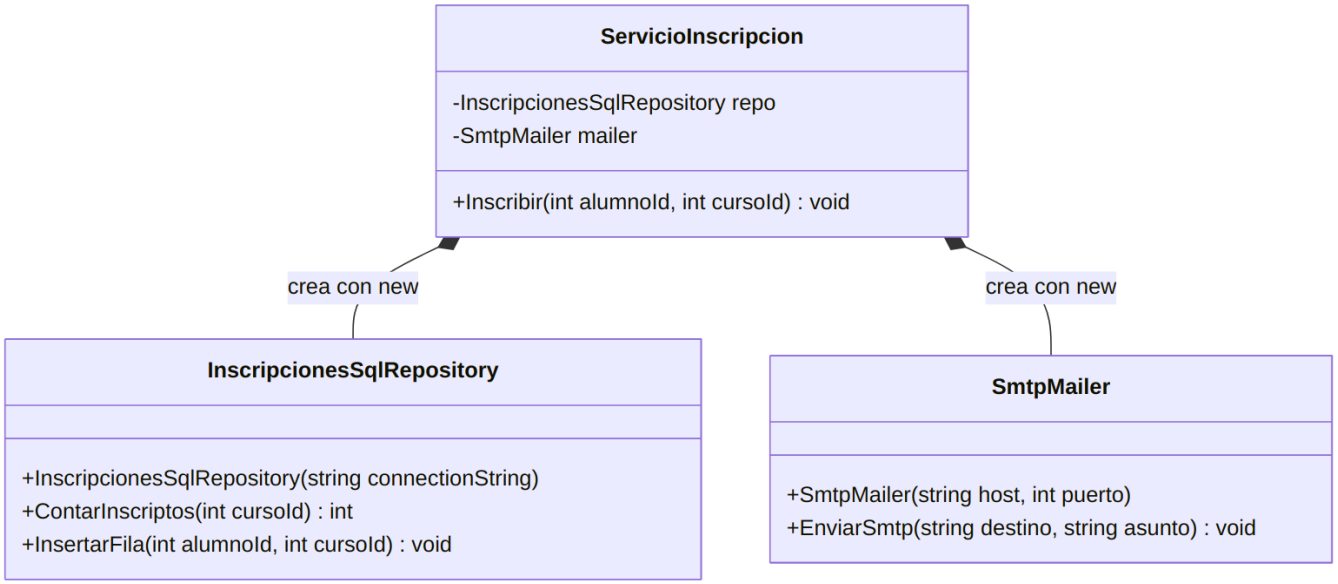
Consigna

1. ¿Qué operaciones puede ejecutar `ReporteNotas` sobre el repositorio que no debería poder ejecutar?
2. Separá `IRepositorio<T>` en interfaces más chicas. El reporte tiene que recibir solo lo que lee y el repositorio de auditoría tiene que exponer solo lo que admite.
3. Las clases que hoy usan el repositorio completo, ¿tienen que cambiar con tu diseño? Mostrá cómo lo resolviste.

Parte 3. Principio de inversión de dependencias

Ejercicio D1. Inscripción a cursos

El servicio de inscripción valida el cupo, registra la inscripción y avisa al alumno por mail.




```
public class ServicioInscripcion
{
    private readonly InscripcionesSqlRepository repo =
        new InscripcionesSqlRepository("Server=prod;Database=campus;...");

    private readonly SmtpMailer mailer =
        new SmtpMailer("smtp.facultad.edu.ar", 587);

    public void Inscribir(int alumnoId, int cursoId)
    {
        if (repo.ContarInscriptos(cursoId) >= 900)
        {
            throw new InvalidOperationException("Curso completo");
        }
```

```
        repo.InsertarFila(alumnoId, cursoId);
        mailer.EnviarSmtp("alumno" + alumnoId + "@facultad.edu.ar", "Inscripción
confirmada");
    }
}
```

Consigna

1. Querés escribir un test para la regla "no se puede inscribir si el curso tiene 900 inscriptos". ¿Qué necesitás tener funcionando para poder correrlo?
2. Mirá los nombres de los métodos `InsertarFila` y `EnviarSmtp`. ¿Qué dicen sobre quién define el vocabulario de esta relación, el módulo de alto nivel o el de bajo nivel?
3. Invertí las dependencias. Indicá en qué capa o proyecto viven las interfaces nuevas y justificá por qué.
4. Escribí el test del punto 1 usando clases falsas hechas a mano (sin librerías de mocking).
5. ¿Dónde se crean ahora las clases concretas? Mostrá ese código.

Ejercicio D2. Fecha límite de entrega

```
public class ValidadorEntrega
{
    public bool EstaEnTermino(DateTime fechaLimite)
    {
        return DateTime.Now <= fechaLimite;
    }
}
```

Consigna

1. Escribí un test que verifique que una entrega hecha a las 23:59 del día límite está en término. ¿Por qué no podés hacerlo de forma confiable?
2. En esta clase no aparece ningún `new`. Aun así tiene una dependencia concreta. ¿Cuál es?
3. Invertí esa dependencia y escribí el test del punto 1.

Parte 4. Ejercicio integrador: corrector automático de trabajos prácticos

Este ejercicio usa los cinco principios. La clase compila la entrega, corre los tests, calcula la nota, la guarda y le avisa al alumno.

```
public class CorrectorTP
{
    public int Corregir(Entrega entrega)
    {
        int nota = 0;

        if (entrega.Lenguaje == "csharp")
        {
```

```
            CompiladorCSharp compilador = new CompiladorCSharp();
            if (!compilador.Compila(entrega.Archivo))
            {
                return 1;
            }
            nota += 4;
        }
        else if (entrega.Lenguaje == "java")
        {
            CompiladorJava compilador = new CompiladorJava();
            if (!compilador.Compila(entrega.Archivo))
            {
                return 1;
            }
            nota += 4;
        }

        EjecutorTestsXUnit tests = new EjecutorTestsXUnit();
        int porcentajeAprobados = tests.Ejecutar(entrega.Archivo);
        nota += porcentajeAprobados * 4 / 100;

        if (DateTime.Now > entrega.FechaLimite)
        {
            nota -= 2;
        }

        SqlConnection conexion = new SqlConnection("...");
        // INSERT de la nota en la base

        SmtpClient smtp = new SmtpClient("smtp.facultad.edu.ar");
        smtp.Send("noreply@facultad.edu.ar", entrega.EmailAlumno, "Nota", "Tu nota
es " + nota);

        return nota;
    }
}
```

Consigna

1. Hacé el diagrama de clases del diseño actual.
2. Listá cada violación que encuentres e indicá qué principio rompe. Hay al menos una por cada principio SOLID.
3. Hacé el diagrama de clases de tu diseño propuesto.
4. Implementá tu diseño en C#. Agregar soporte para entregas en Python no puede requerir modificar ninguna clase existente.
5. Escribí un test del cálculo de la nota que no use base de datos, compilador real, servidor de mail ni la fecha del sistema.
6. Si en tu diseño aparece una jerarquía o una interfaz de compiladores, escribí una prueba de contrato (como la del ejercicio L2) que verifique que todas las implementaciones son sustituibles entre sí.